

LMER Model Predictions

Erik Larsen

2026-05-11

Overview

This markdown is:

- a rough proxy for a tutorial on interpreting linear mixed model outputs
- a light introduction to golf scoring systems

The markdown shows:

- the linear mixed model used to measure my performance for each golf round and predict my performance in the next golf round

The model does this by incorporating my skill level and where I've played in the past

Introduction

Skill level is officially determined by the **USGA** as a **Handicap Index**: how many strokes, on average, a player takes to complete a round at a given course with a given difficulty relative to that course's average:

- a player with a **10.0 Handicap Index** is expected, on average, to take **82 strokes** to complete a round at a course where the average number of strokes has been determined to be **72**

Gross Score is a total number of strokes taken by a given player for any individual hole

- this is extrapolated across each of 18 holes, so while **Gross Score** can mean a total of strokes for a hole *or* a total of strokes for a round, in this markdown, **it is per round**
- the average **Gross Score** over a player's best **8 rounds** *from their last 20* is used to determine their **Handicap Index**

Thus, this markdown shows the model used to predict the opposite:

- given my **Handicap Index** and **Gross Scores** at previous courses of varying difficulty in the past,
 - **what will be my next Gross Score?**

Note that I use the terms, **strokes**, and, **shots**, interchangeably, though putts are not connoted as **shots**

Gather Data and Format Scores

```
library(golf)
library(tidyverse)
library(lme4)
library(DBI)
```

Attach Packages

Gather Scores Gather and format from the database

```
## connect to the db
con <- golf::get_db_connection()

scores <- DBI::dbGetQuery(conn = con, statement = paste0(
  "SELECT DISTINCT r.*, c.par, c.course_rating, c.slope FROM rounds r
  INNER JOIN courses c
  ON c.tees = r.tees
  AND c.course_name = r.course_name
  AND c.hole = r.hole
  INNER JOIN players p
  ON r.GHIN = p.GHIN
  AND r.handicap_index = p.handicap_index
  AND r.date = p.date;"
)) |>
  dplyr::mutate(date = lubridate::as_date(date), # convert strings to date's
               hole = gsub(hole, pattern = 'hole_', replacement = '') |> # enforce hole order
               as.numeric()) |>
  dplyr::relocate(par, .after = hole) |>
  dplyr::relocate(course_rating, .after = tees) |>
  dplyr::group_by(date) |>
  dplyr::arrange(desc(date), hole) |>
  dplyr::ungroup()
```

Compute Metrics Compute standard metrics

Scoring Metrics Show Per-Round Gross Scores, Net Scores and Handicap Index

```
scoring_metrics <- scores_sum |>
  dplyr::select(`Handicap Index`, `Gross Score`, `Net Score`)
head(scoring_metrics |>
  dplyr::arrange(desc(date)))
```

```
## # A tibble: 6 x 7
## # Groups:   GHIN, date, date_course, course_rating [6]
##   GHIN date      date_course      course_rating `Handicap Index` `Gross Score`
##   <int> <date>    <chr>                <dbl>         <dbl>         <dbl>
## 1 10526424 2026-04-26 "2026-04-26\nDell Urich\n11"      68.5           11           92
## 2 10526424 2026-04-19 "2026-04-19\nRandolph North\n11"  71.7           11           91
## 3 10526424 2026-04-05 "2026-04-05\nRandolph North\n11"  71.7           11           88
## 4 10526424 2026-03-29 "2026-03-29\nDell Urich\n10"      70.3           10           82
```

## 5	10526424	2026-03-08	"2026-03-08\nRandolph North\n10"	71.7	10	84
## 6	10526424	2026-02-22	"2026-02-22\nDell Urich\n10.1"	68.5	10.1	92

Stroke Metrics In golf, every hole has a **par**– an average number of strokes taken to get the ball in the hole

There are (almost always), three different **pars** on every course:

- par 3s
- par 4s
- par 5s

A shorthand to determine how a golfer performed across holes is to rate how many strokes *relative to par* they were (-1, -2, +1, +2, etc)

These numbers are given names:

- 0 = par
- -2 = eagle
- -1 = birdie
- +1 = bogey
- +2 = double bogey

I quantify these below:

```
stroke_metrics <- scores_sum |>
  dplyr::select(`doubles+`, bogies, pars, birdies)
head(stroke_metrics |>
  dplyr::arrange(desc(date)))
```

```
## # A tibble: 6 x 8
## # Groups:   GHIN, date, date_course, course_rating [6]
##      GHIN date      date_course      course_rating `doubles+` bogies  pars birdies
##      <int> <date>      <chr>          <dbl>      <int>  <int> <int>  <int>
## 1 10526424 2026-04-26 "2026-04-26\nDell Urich\n11"      68.5         6     7     4     1
## 2 10526424 2026-04-19 "2026-04-19\nRandolph North\n11"    71.7         4     8     6     0
## 3 10526424 2026-04-05 "2026-04-05\nRandolph North\n11"    71.7         3     9     6     0
## 4 10526424 2026-03-29 "2026-03-29\nDell Urich\n10"      70.3         4     5     7     2
## 5 10526424 2026-03-08 "2026-03-08\nRandolph North\n10"    71.7         3     6     9     0
## 6 10526424 2026-02-22 "2026-02-22\nDell Urich\n10.1"    68.5         6     7     5     0
```

Around-the-Green Metrics Chips: + strokes around the green taken to get onto the green

Putts: + strokes taken with the putter on the green

Avg GIR putts: + Average # of putts on holes where the ball was hit on to the green within 2 strokes of par (green in regulation, GIR)

UpDown% (aka Scramble%): + # of holes without a GIR but par was made / # of holes without a GIR

```
atg_metrics <- scores_sum |>
  dplyr::select(chips, `chips+putts`, `UpDown%`, putts, `Avg GIR putts`)
head(atg_metrics |>
  dplyr::arrange(desc(date)))
```

```
## # A tibble: 6 x 9
## # Groups:   GHIN, date, date_course, course_rating [6]
##       GHIN date       date_course       course_rating chips `chips+putts` `UpDown%` p
##       <int> <date>       <chr>                <dbl> <dbl>         <dbl> <dbl> <dbl>
## 1 10526424 2026-04-26 "2026-04-26\nDell Urich\n11"          68.5    19            55    15.4
## 2 10526424 2026-04-19 "2026-04-19\nRandolph North\n11"        71.7    20            56    15.4
## 3 10526424 2026-04-05 "2026-04-05\nRandolph North\n11"        71.7    21            52     20
## 4 10526424 2026-03-29 "2026-03-29\nDell Urich\n10"          70.3    15            46     20
## 5 10526424 2026-03-08 "2026-03-08\nRandolph North\n10"        71.7    12            45     30
## 6 10526424 2026-02-22 "2026-02-22\nDell Urich\n10.1"          68.5    18            52    16.7
```

Ball Striking Approach and tee accuracy

GIR (green in regulation): + hole where the ball was hit on to the green within 2 strokes of par

FIR (fairway in regulation): + hole where the ball was hit on to the fairway from the tee box (only on par 4's and par 5's)

Iron FIR: + hole where an iron was used off the tee for a FIR

Driver FIR: + hole where driver was used off the tee for a FIR

```
ball_striking_metrics <- scores_sum |>
  dplyr::select(GIRs, `GIR%`, `Par 3 GIRs`,
                FIRs, `FIR%`, `Iron FIRs`, `Iron FIR%`,
                `Driver FIRs`, `Driver FIR%`)
head(ball_striking_metrics |>
  dplyr::arrange(desc(date)))
```

```
## # A tibble: 6 x 13
## # Groups:   GHIN, date, date_course, course_rating [6]
##       GHIN date       date_course       course_rating GIRs `GIR%` `Par 3 GIRs` FIRs
##       <int> <date>       <chr>                <dbl> <int> <dbl>         <dbl> <int>
## 1 10526424 2026-04-26 "2026-04-26\nDell Urich\n11"          68.5     3    16.7           0     5
## 2 10526424 2026-04-19 "2026-04-19\nRandolph North\n11"        71.7     4    22.2           1     3
## 3 10526424 2026-04-05 "2026-04-05\nRandolph North\n11"        71.7     3    16.7           1     6
## 4 10526424 2026-03-29 "2026-03-29\nDell Urich\n10"          70.3     7    38.9           1     5
## 5 10526424 2026-03-08 "2026-03-08\nRandolph North\n10"        71.7     7    38.9           2     2
## 6 10526424 2026-02-22 "2026-02-22\nDell Urich\n10.1"          68.5     3    16.7           1     7
```

Shot Quality Yardage and accuracy on tracked shots

```
## connect to the db
con <- golf::get_db_connection()

stroke_quality <- DBI::dbGetQuery(conn = con,
                                  statement = paste0("SELECT DISTINCT * FROM club_metrics;")) |>
  dplyr::mutate(date = lubridate::as_date(date),
               hole = gsub(hole, pattern = 'hole_', replacement = ''),
               hole = as.integer(hole)) |>
  dplyr::mutate(dplyr::across(dplyr::contains("yds"), ~as.integer(.x))) |>
  dplyr::rename(course = course_name) |>
  dplyr::group_by(date, hole, stroke) |>
  dplyr::arrange(date, hole, stroke)
```

```
head(stroke_quality |>
  dplyr::arrange(desc(date)))
```

```
## # A tibble: 6 x 14
## # Groups:   date, hole, stroke [6]
##   course      date      tees hole   par gross stroke lie   club yds_to_target yds_traveled on_targ
##   <chr>      <date>      <chr> <int> <int> <int> <int> <chr> <chr>      <int>      <int> <chr>
## 1 Dell Urich 2026-04-26 combo     1     4     5     1 tee   D        270      229 no
## 2 Dell Urich 2026-04-26 combo     1     4     5     2 rough 9        160      174 no
## 3 Dell Urich 2026-04-26 combo     1     4     5     3 rough GW        35       40 yes
## 4 Dell Urich 2026-04-26 combo     2     3     5     1 tee   9        160      155 no
## 5 Dell Urich 2026-04-26 combo     2     3     5     2 rough SW         55       30 yes
## 6 Dell Urich 2026-04-26 combo     3     5     4     1 tee   D        270      298 yes
```

Fit a LMER Model

Fit a lmer model to capture repeated measurements of Gross Score predicted by Handicap Index, course_rating, and time (days).

- Every course has a rating; the Handicap Index calculation factors in these ratings
- Center the course_rating and Handicap Index variables at their mean to make interpreting intercept and slope estimates more meaningful
- Include random intercepts and random slopes of course and course_rating, given a Handicap Index

Fit a model to the data

```
gross_lmer <- lme4::lmer(

  data = scores_sum |>
    dplyr::ungroup() |>
    dplyr::mutate(

      course_rating = course_rating - mean(course_rating),

      course = gsub(date_course,
                    pattern = '[0-9]|\\-|\\\\\\n|\\\\\\.',
                    replacement = ''), # extract the course names

      `Handicap Index` = -`Handicap Index` - mean(-`Handicap Index`),
      days = as.numeric(as.Date(date) - min(as.Date(date))) + 1,
      units = 'days')

    ) |> # create a 'days' metric starting at the first day joining the club

  dplyr::relocate(days, .after = date),

  formula =
    `Gross Score` ~
    `Handicap Index`*course_rating +
    days +
    (1 + `Handicap Index`|course) #+ # random intercepts and random slopes for Gross Score at a given c
    # (1 + `Handicap Index`|course_rating) # random intercepts and random slopes for Gross Score at a g
    ) # somewhat redundant
```

LMER Model Summary

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: `Gross Score` ~ `Handicap Index` * course_rating + days + (1 + `Handicap Index` | course)
## Data: dplyr::relocate(dplyr::mutate(dplyr::ungroup(scores_sum), course_rating = course_rating -
## replacement = ""), `Handicap Index` = -`Handicap Index` - mean(-`Handicap Index`), days = as.Date(
## min(as.Date(date)) + 1, units = "days")), days, .after = date)
##
## REML criterion at convergence: 180
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.58890 -0.71861 -0.01928  0.60278  1.62038
##
## Random effects:
##   Groups   Name                Variance Std.Dev. Corr
##   course   (Intercept)          84.954   9.217
##           `Handicap Index`    7.923    2.815   -1.00
##   Residual                    10.752   3.279
## Number of obs: 32, groups:  course, 5
##
## Fixed effects:
##              Estimate Std. Error t value
## (Intercept)    97.12733    5.03273  19.299
## `Handicap Index` -1.03732    1.58027  -0.656
## course_rating    4.79617    1.31356   3.651
## days           -0.02654    0.01028  -2.582
## `Handicap Index`:course_rating -3.60930    0.75566  -4.776
##
## Correlation of Fixed Effects:
##              (Intr) `HInd` crs_rt days
## `HndcpIndx` -0.715
## course_rtnng  0.469 -0.334
## days         -0.496 -0.107 -0.576
## `HIndx`:cr_ -0.359  0.315 -0.776  0.300
## optimizer (nloptwrap) convergence code: 0 (OK)
## boundary (singular) fit: see help('isSingular')
```

```
golf::export_lm_round_predictions(
  model = gross_lmer,
  scores_sum = scores_sum |>
    dplyr::mutate(date = as.character(date))
)
```

Export Model Results to db

Model Interpretations

Fixed Effects

Gross Score Intercept The model's estimated average *first* Gross Score ((Intercept) Estimate of Fixed effects) at my average Handicap Index and average course_rating at Arizona National (default reference course) is **97.13**.

My average Gross Score, however, is **85.59**.

Handicap Index For every additional Handicap Index point improvement (lower) than my average Handicap Index, my expected Gross Score decreases by **1.04** strokes.

- This makes sense because Gross Score is used to directly determine Handicap Index and is positively correlated:
 - high Gross Score = high Handicap Index
 - In other words, a player with better skill will have a lower Handicap Index
 - * An example: a player with a **0** index means they average **par** (the course average) for an entire round, across rounds, while a worse player (who averages above **par**, the course average) will have a higher Handicap Index
 - * Handicap Index corrects for skill-level
 - The effect is not significant (**t value = -0.66**; significance : $\text{abs}(\text{t value}) > 1$)
 - Again, Handicap Index is a metric *directly derived from* Gross Score
 - * I'm unsure how many strokes (Gross Score) index points *should* be worth! **1? More?**
 - * Does it vary by skill, or is it uniform?

Course Rating For every additional course_rating point (aka, a stroke) greater than the average course_rating (~69-70 strokes in this dataset), Gross Score increases by **4.8** strokes (it decreases).

- This also makes sense: harder courses should yield higher Gross Scores

Time (days) For every additional day in time, my Gross Score drops by **-0.03** strokes

- While this seems tiny, extrapolating days to months or weeks, this becomes very evident (**-0.9** strokes per month; **-10.95** strokes per year)
- Linear extrapolation in this sense is misleading: there will be a limit to lowering Gross Score and there will also be variation in the process
- But this effect is strongly significant (**t value = -2.58**) and appears to be the primary driver of the trend

Handicap Index*Course Rating Interaction As course_rating increases by 1 stroke above average, the effect of Handicap Index on my expected Gross Score is **3.61** strokes less than what the two effects would contribute independently.

- In other words: hard courses already impose a big penalty, so the *extra* penalty by playing worse than average is smaller
- Related: easier courses impose a larger *extra* penalty by playing worse than average
- Related: harder courses add a larger bonus when playing better than average
- Related: easier courses add a smaller bonus when playing better than average

Random Effects

Course, Handicap Index, and Time These `courses` vary in their difficulty, independent of player skill (`Handicap Index`), by $\sim \pm 9.22$ strokes. This value is the **Random effects Std. Dev. (Intercept) from the model summary**– the Std.Dev. of the course-level random intercepts, representing how much each course shifts my baseline expected `Gross Score` up or down relative to the overall average, *even after accounting for `course_rating`*.

- I play different courses much differently

Interestingly, `courses` also differ slightly in how sensitive they are to my `Handicap Index`, with a random-slope standard deviation of ± 2.81 strokes per index point.

While there is a fair amount of variability in `Gross Score` driven by the `course`, there is also just a large amount of variability in `Gross Score`, overall: **3.28**. This is the **Random effects Residual Std.Dev.** from the model summary.

Predict the Next Round

Predict the next round's `Gross Score` according to the model

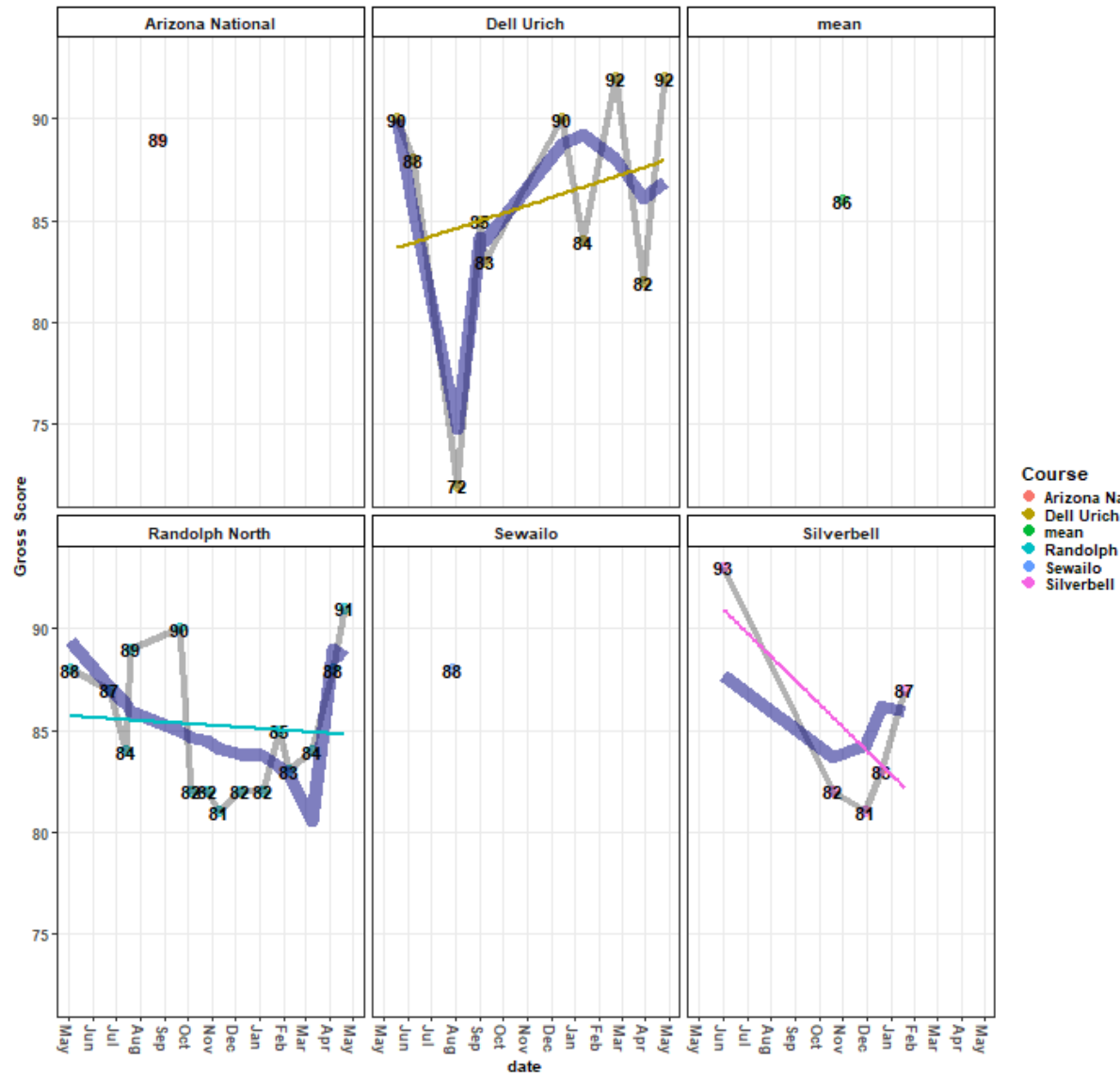
```
## show the model-predicted gross score for the upcoming round, rounded to the nearest stroke
stats::predict(object = gross_lmer, newdata = new_df, allow.new.levels = T) |>
  as.numeric() %>%
  round(., 0)
```

Show Prediction

```
## [1] 81
```

Plot the Model

Linear Mixed Effects Regression (LMER) Model trained on Gross Scores as of 2026-05-11



Model by Course

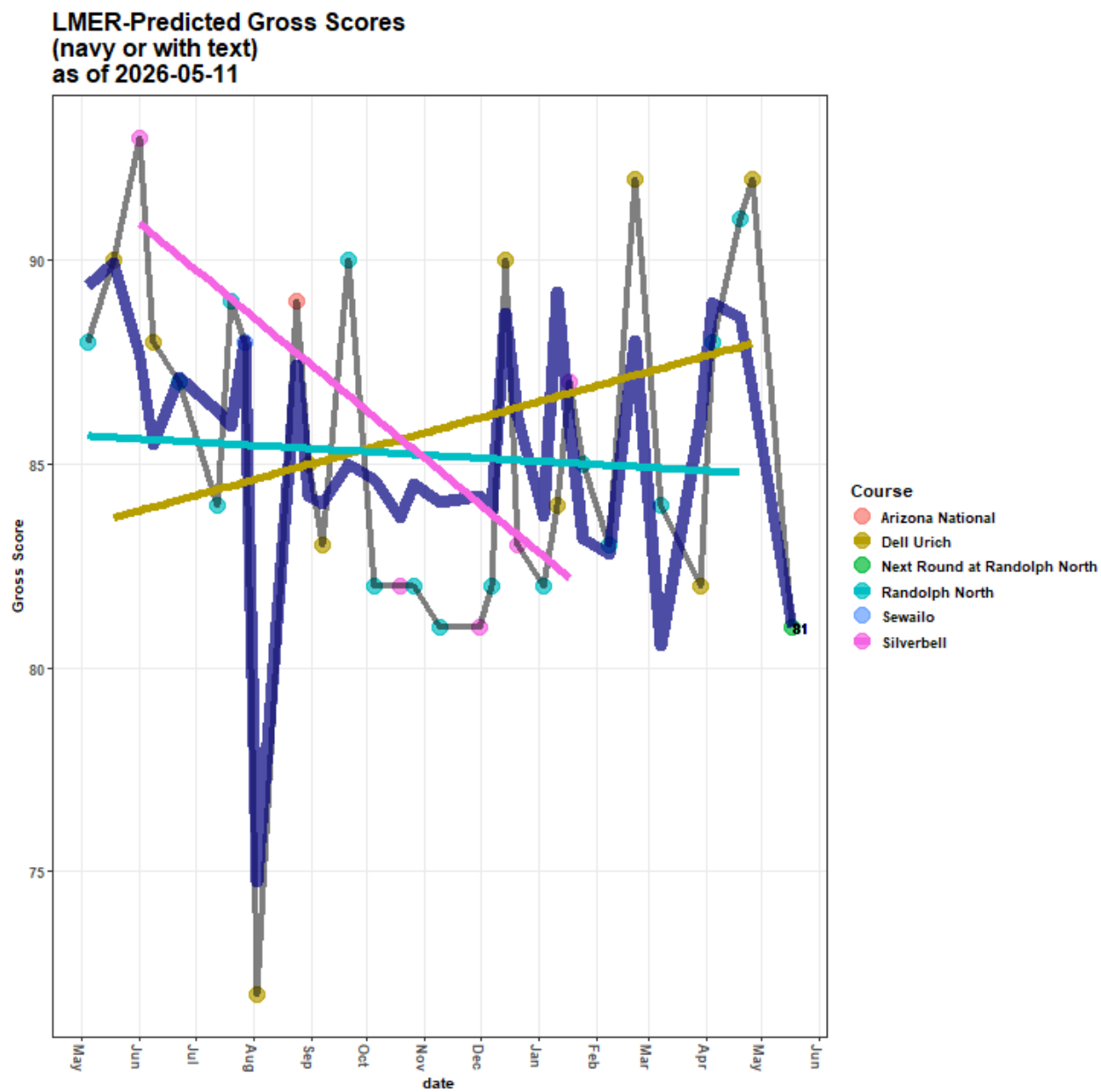
The model is a random intercept, random slope linear mixed-effects regression (LMER) model.

In this case, that means **Gross Score** varies for each course at a given **Handicap Index** in its deviation from the overall mean **Gross Score** (navy blue line) over time: **Silverbell**, **Randolph North**, and **Dell Ulrich** have their own average **Gross Scores** (intercepts) and slopes (change in **Gross Score** over time)– notice how the line for each course has a different slope, starting at a different y-intercept

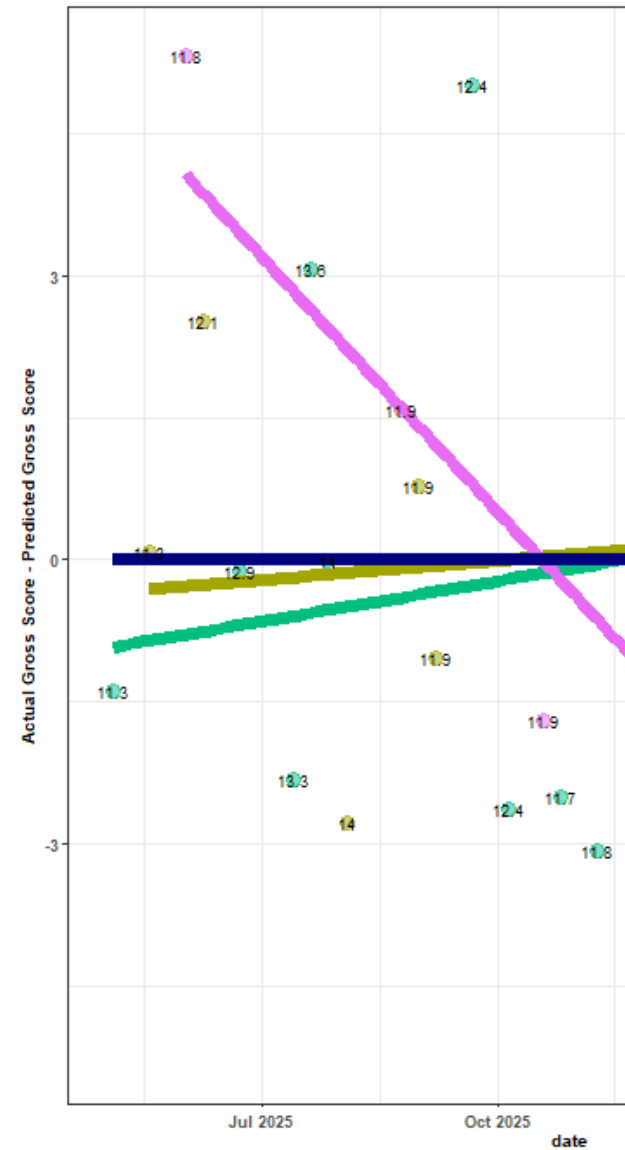
- The blue line is the model's overall fit of the **Gross Score**, accounting for course, course_rating, Handicap Index, and days (time)
- **Silverbell's** line represents the relationship between **Gross Score**, course_rating, Handicap Index, and days (date/time) at **Silverbell**
- **Randolph's** line represents the relationship between **Gross Score**, course_rating, Handicap Index, and days (date/time) at **Randolph North**
- **Dell Ulrich's** line represents the relationship between **Gross Score**, course_rating, Handicap Index, and days (date/time) at **Dell Ulrich**

Model Predictions These predictions are **not** historical– they are current:

- i.e. the model did *not* predict anything close to the **72** in August 2025



Performance vs Prediction
from 2025-05-04 - present
as of 2026-05-11



Actual Gross vs Predicted Gross (performance vs prediction)

This plot of residuals reveals the Actual Gross Score relative to the Predicted Gross Score over time, color-coded by course, and annotated by Handicap Index at the time of the round.

- the navy blue line represents the dividing line between over/under performing where:
 - scores **above** the line = I performed **worse** than the model's prediction
 - scores **below** the line = I performed **better** than the model's prediction
- Randolph North, Silverbell, and Dell Urich each have lines representing the trend of actual Gross Scores compared to predicted Gross Scores at each respective course
 - I more often score better/lower at Randolph North than the model predicts, though, on average, these are closest to the model's predictions
 - * This might reveal that these rounds are contributing more weight to the model if they are not simply more reflective of the overall trend

- * This could also reveal that I score more consistently at this course than others
- * This could also mean that, given the downward trend, the course is easier than ratings suggest
- At Silverbell and Randolph North, I have been outperforming the model and scoring better over time
- The variability at Dell Ulrich is substantial, with one large outlier overperformance (~ -15), and one moderate outlier underperformance ($\sim +6$) re-shaping the slope in the opposite direction: I have been getting worse at Dell Ulrich over time
- * Even with more subsequent sample, this could reveal that I struggle to shoot low Gross Scores at Dell Ulrich, despite its easier course rating, and any effect of time
- * Other latent variables may contribute to this variability, such as course/weather/event conditions

